



Универзитет „Св. Кирил и Методиј“ во Скопје
ФАКУЛТЕТ ЗА ИНФОРМАТИЧКИ НАУКИ И
КОМПЈУТЕРСКО ИНЖЕНЕРСТВО

ФАКУЛТЕТ ЗА ИНФОРМАТИЧКИ НАУКИ И КОМПЈУТЕРСКО
ИНЖЕНЕРСТВО

ЕЛАБОРАТ ПО ПРЕДМЕТОТ

Континуирана интеграција и испорака

2025/26

Task Manager Application

FastAPI · React (Vite/TypeScript) · PostgreSQL · Docker
Kubernetes · CI/CD

Професор:

проф. д-р Панче Рибарски

Асистент:

асс. м-р Стефан Андонов

Предмет:

Континуирана интеграција и
испорака-2025/26

Студент:

Стефан Везенкоски

Индекс:

233152

Линкови:

GitHub: <https://github.com/stefanvezenkoski/task-manager-app>

DockerHub: <https://hub.docker.com/repositories/stefanvezenkoski>

Скопје, 01.08.2026

Содржина

1	Вовед / Опис на апликацијата	1
1.1	Технологии	1
1.2	Архитектура на апликацијата	1
2	Докеризација	3
2.1	Dockerfile за backend	3
2.2	Dockerfile за frontend	3
2.3	Docker Compose	5
3	CI/CD Pipeline	7
4	Kubernetes манифести	10

Вовед / Опис на апликацијата

Task Manager Application е full-stack веб апликација за менаџирање задачи (to-do list), изработена како практична основа за проектната задача по предметот **Континуирана интеграција и испорака**. Апликацијата овозможува креирање, прегледување, ажурирање, означување како завршени и бришење на задачи, преку REST API поврзан со релациона база на податоци.

1.1 Технологии

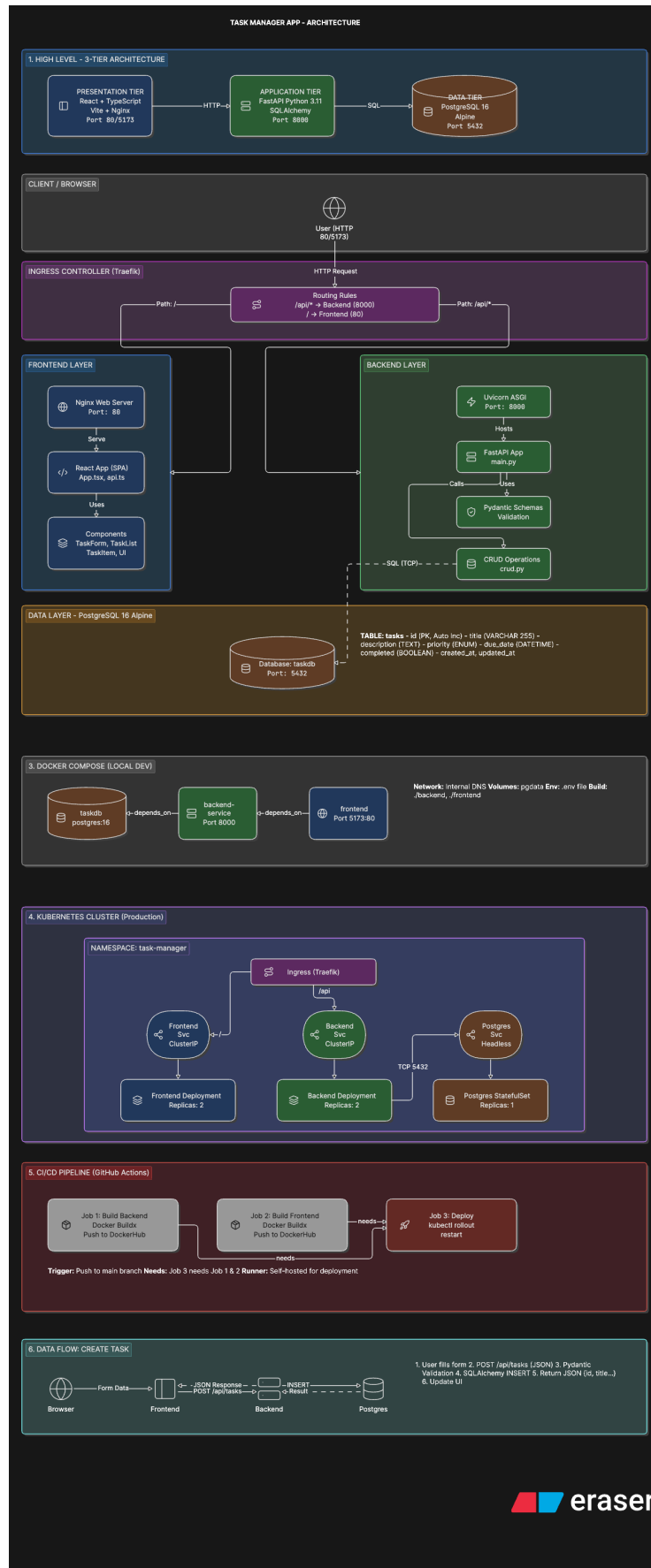
- **Backend:** FastAPI (Python 3.11), SQLAlchemy ORM, Pydantic шеми за валидација
- **Frontend:** React 18 + TypeScript, Vite build алатка
- **База на податоци:** PostgreSQL 16
- **Контејнеризација:** Docker, Docker Compose
- **Оркестрација:** Kubernetes (k3d кластер, Traefik Ingress Controller)
- **CI/CD:** GitHub Actions, DockerHub, self-hosted runner за CD

Изворниот код е достапен на јавен GitHub репозиториум: <https://github.com/stefanv ezenkoski/task-manager-app>.

1.2 Архитектура на апликацијата

Апликацијата е поделена на три независни сервиси кои комуницираат преку HTTP и SQL мрежни протоколи, следејќи класична 3-слојна (presentation / application / data) архитектура.

Секој HTTP барање од корисникот преминува низ Ingress контролерот (Traefik), кој го рутира сообраќајот кон соодветниот Kubernetes Service, а оттаму кон подот што ја извршува апликацијата. Пониската слика ја прикажува целосната архитектура — од локален development (Docker Compose) до продукциско опкружување (Kubernetes):

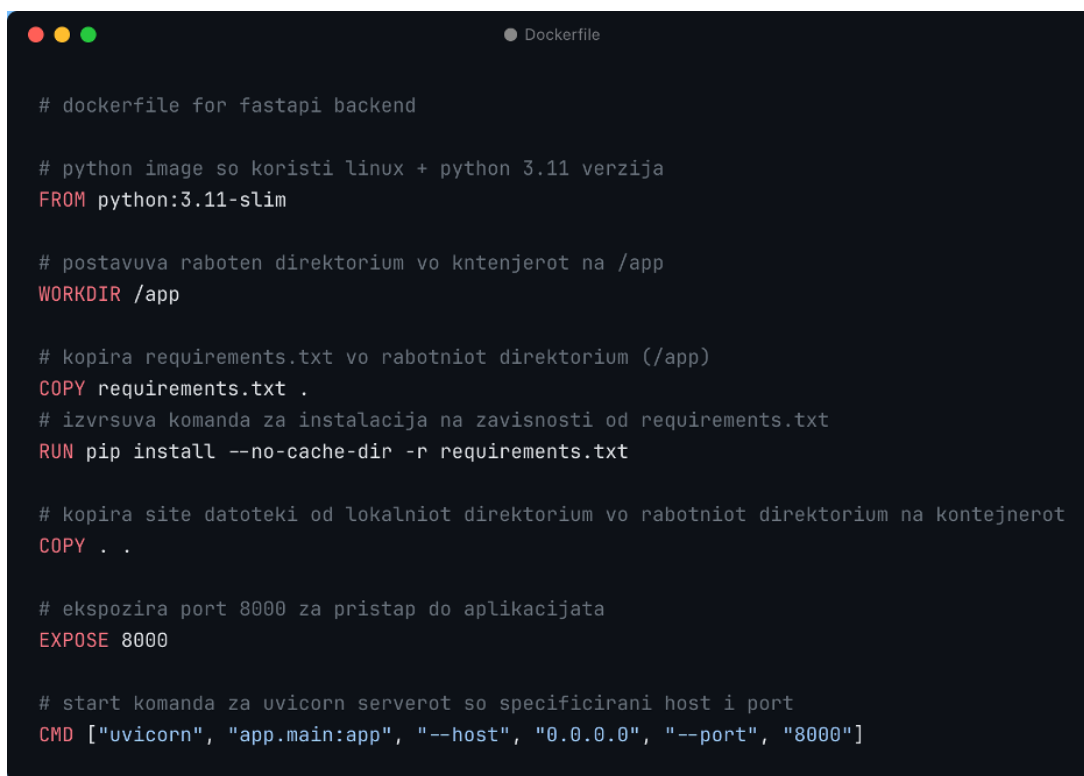


Слика 1.1: Целосна архитектурна шема (high-level, Ingress, Kubernetes кластер, CI/CD pipeline и data flow)

Докеризација

2.1 Dockerfile за backend

Backend-от се докеризира со лесна Python слика (python:3.11-slim), се инсталираат dependencies преку requirements.txt, и серверот се стартува со uvicorn на порт 8000:

A screenshot of a code editor window titled 'Dockerfile'. The code is written in a dark theme with syntax highlighting. It defines a Docker image for a FastAPI backend. The instructions include: setting the base image to python:3.11-slim, setting the working directory to /app, copying requirements.txt, installing dependencies with pip, copying the application code, exposing port 8000, and setting the command to run uvicorn.

```
# dockerfile for fastapi backend

# python image so koristi linux + python 3.11 verzija
FROM python:3.11-slim

# postavuva raboten direktorium vo kntenjerot na /app
WORKDIR /app

# kopira requirements.txt vo rabotniot direktorium (/app)
COPY requirements.txt .
# izvrsuva komanda za instalacija na zavisnosti od requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

# kopira site datoteki od lokalniot direktorium vo rabotniot direktorium na kontejnerot
COPY . .

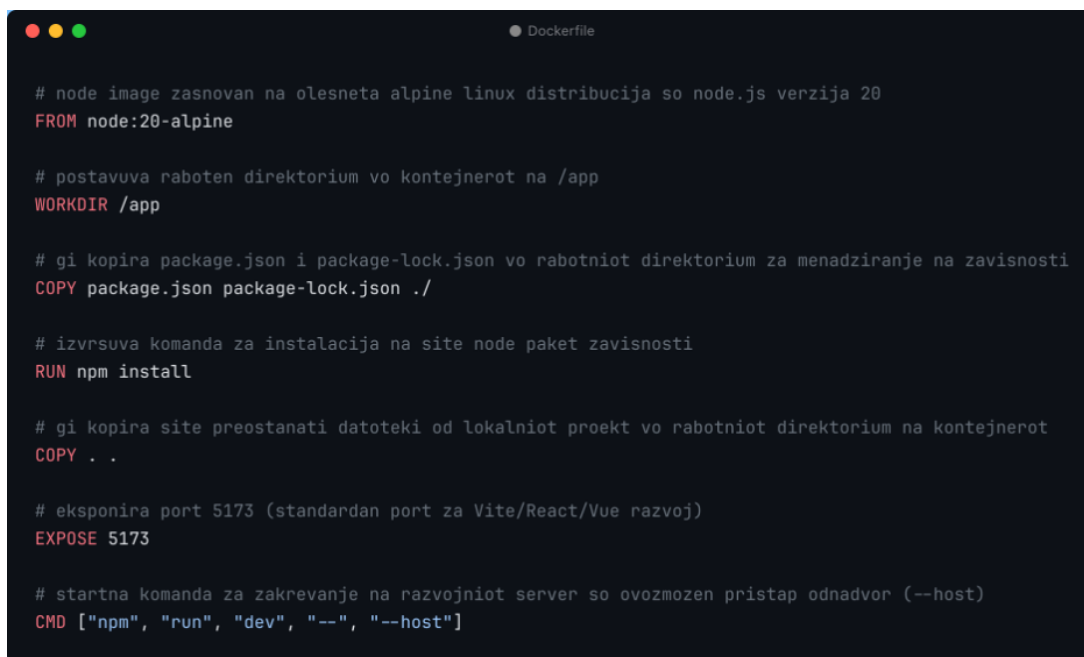
# ekspozira port 8000 za pristap do aplikacijata
EXPOSE 8000

# start komanda za uvicorn serverot so specificirani host i port
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Слика 2.1: Dockerfile за FastAPI backend

2.2 Dockerfile за frontend

Првичната верзија на frontend Dockerfile-от користеше npm run dev (Vite development server). Ова работеше локално преку Docker Compose, но при деплојмент во Kubernetes зад Ingress контролер, dev серверот фрлаше WebSocket грешки (Hot Module Replacement пробува да отвори конекција која Ingress-от не ја проследува правилно), а страницата остануваше празна во browser-от.

A screenshot of a code editor showing a Dockerfile. The title bar says "Dockerfile". The code is written in a dark theme with syntax highlighting. It starts with a comment about the base image being Alpine Linux with Node.js 20. The instructions include: FROM node:20-alpine, WORKDIR /app, COPY package.json package-lock.json ./, RUN npm install, COPY . ., EXPOSE 5173, and CMD ["npm", "run", "dev", "--", "--host"].

```
# node image zasnovan na olesneta alpine linux distribucija so node.js verzija 20
FROM node:20-alpine

# postavlja raboten direktorium vo kontejnerot na /app
WORKDIR /app

# gi kopira package.json i package-lock.json vo rabotniot direktorium za menadziranje na zavisnosti
COPY package.json package-lock.json ./

# izvrsuva komanda za instalacija na site node paket zavisnosti
RUN npm install

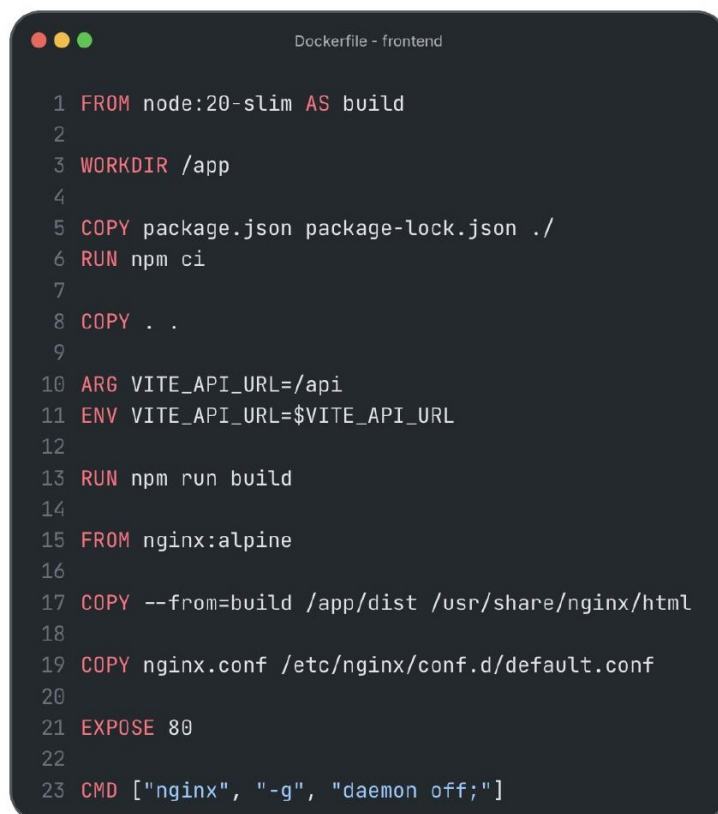
# gi kopira site preostanati datoteki od lokalniot proekt vo rabotniot direktorium na kontejnerot
COPY . .

# eksponira port 5173 (standardan port za Vite/React/Vue razvoj)
EXPOSE 5173

# startna komanda za zakrevanje na razvojniot server so ovozmozen pristap odnadvor (--host)
CMD ["npm", "run", "dev", "--", "--host"]
```

Слика 2.2: Првична (development) верзија на frontend Dockerfile-от

Решение: **multi-stage build** — прв чекор build-ира статички production фајлови со `npm run build`, втор чекор ги сервира преку `nginx` на порт 80. Ова е стандардна пракса за production React апликации и целосно го отстранува WebSocket проблемот.

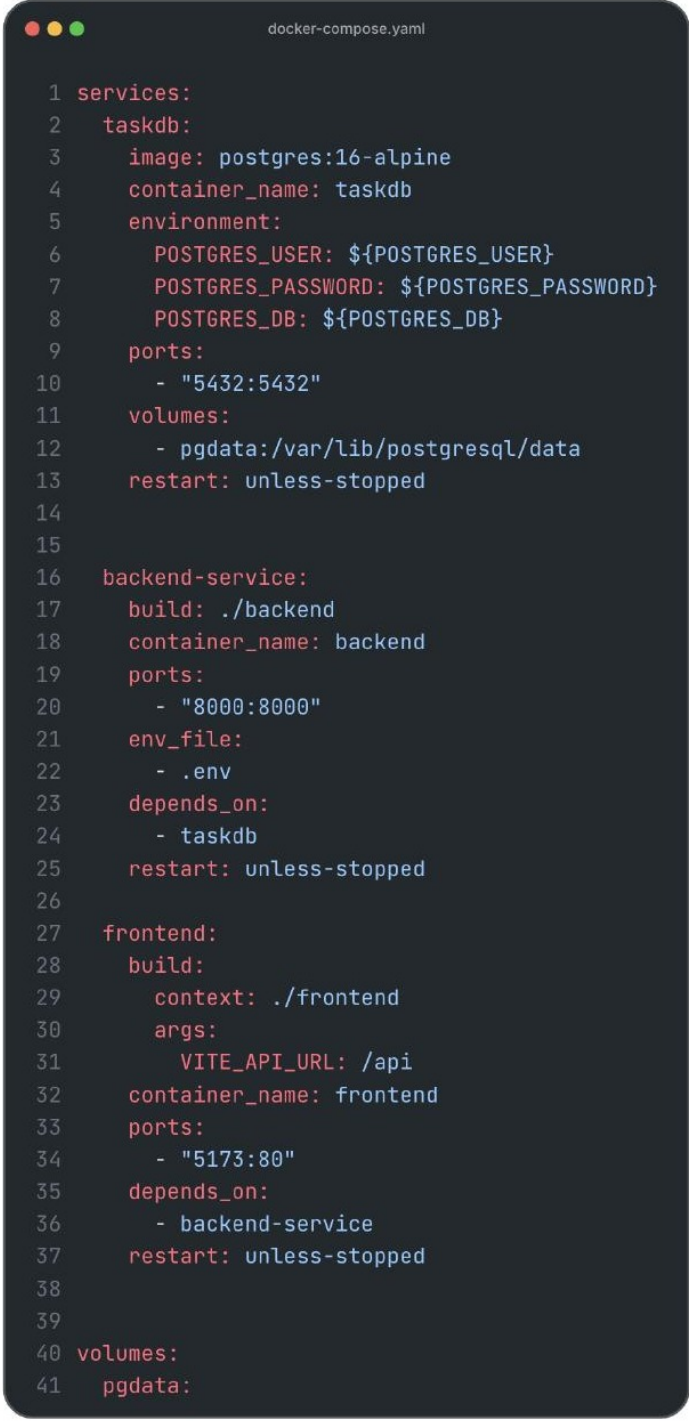
A screenshot of a code editor showing a multi-stage Dockerfile. The title bar says "Dockerfile - frontend". The code is written in a dark theme with syntax highlighting. It has two stages. Stage 1 (build) starts with FROM node:20-slim AS build, sets WORKDIR to /app, copies package files, runs npm ci, copies the rest of the code, sets environment variables for VITE_API_URL, runs npm run build, and then FROM nginx:alpine. Stage 2 starts with COPY --from=build /app/dist to the nginx html directory, copies the nginx.conf file, sets EXPOSE 80, and sets CMD to ["nginx", "-g", "daemon off;"].

```
1 FROM node:20-slim AS build
2
3 WORKDIR /app
4
5 COPY package.json package-lock.json ./
6 RUN npm ci
7
8 COPY . .
9
10 ARG VITE_API_URL=/api
11 ENV VITE_API_URL=$VITE_API_URL
12
13 RUN npm run build
14
15 FROM nginx:alpine
16
17 COPY --from=build /app/dist /usr/share/nginx/html
18
19 COPY nginx.conf /etc/nginx/conf.d/default.conf
20
21 EXPOSE 80
22
23 CMD ["nginx", "-g", "daemon off;"]
```

Слика 2.3: Финална (production) верзија: multi-stage Dockerfile со nginx

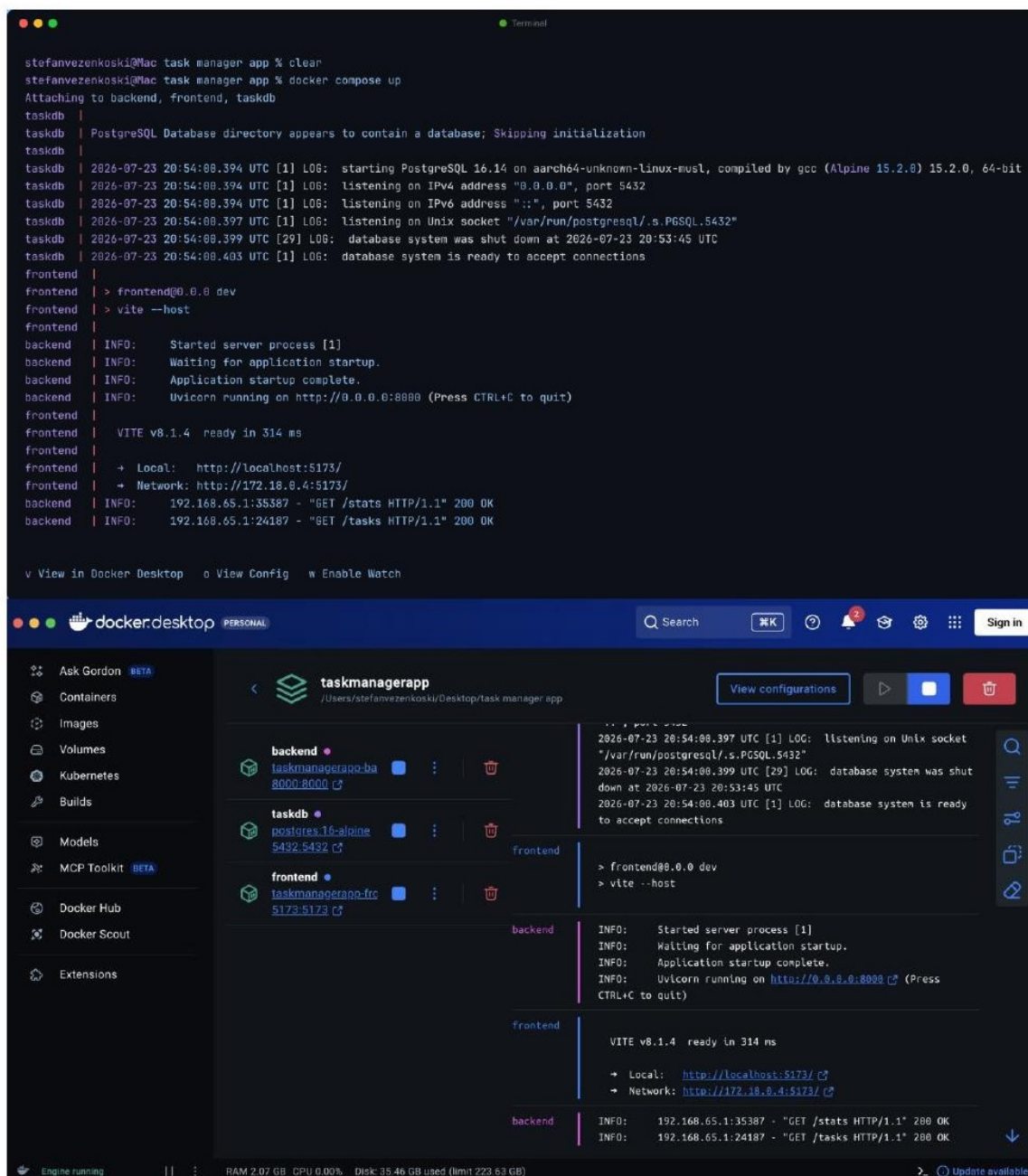
2.3 Docker Compose

Трите сервиси (frontend, backend, taskdb) се оркестрирани заедно преку docker-compose.yml, со внатрешна DNS мрежа, volume за персистентност на податоците, и .env фајл за конфигурација:



```
1 services:
2   taskdb:
3     image: postgres:16-alpine
4     container_name: taskdb
5     environment:
6       POSTGRES_USER: ${POSTGRES_USER}
7       POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
8       POSTGRES_DB: ${POSTGRES_DB}
9     ports:
10      - "5432:5432"
11     volumes:
12      - pgdata:/var/lib/postgresql/data
13     restart: unless-stopped
14
15
16   backend-service:
17     build: ./backend
18     container_name: backend
19     ports:
20      - "8000:8000"
21     env_file:
22      - .env
23     depends_on:
24      - taskdb
25     restart: unless-stopped
26
27   frontend:
28     build:
29       context: ./frontend
30     args:
31       VITE_API_URL: /api
32     container_name: frontend
33     ports:
34      - "5173:80"
35     depends_on:
36      - backend-service
37     restart: unless-stopped
38
39
40 volumes:
41   pgdata:
```

Слика 2.4: docker-compose.yml



Слика 2.5: docker compose up: сите три сервиси стартуваат и комуницираат успешно

CI/CD Pipeline

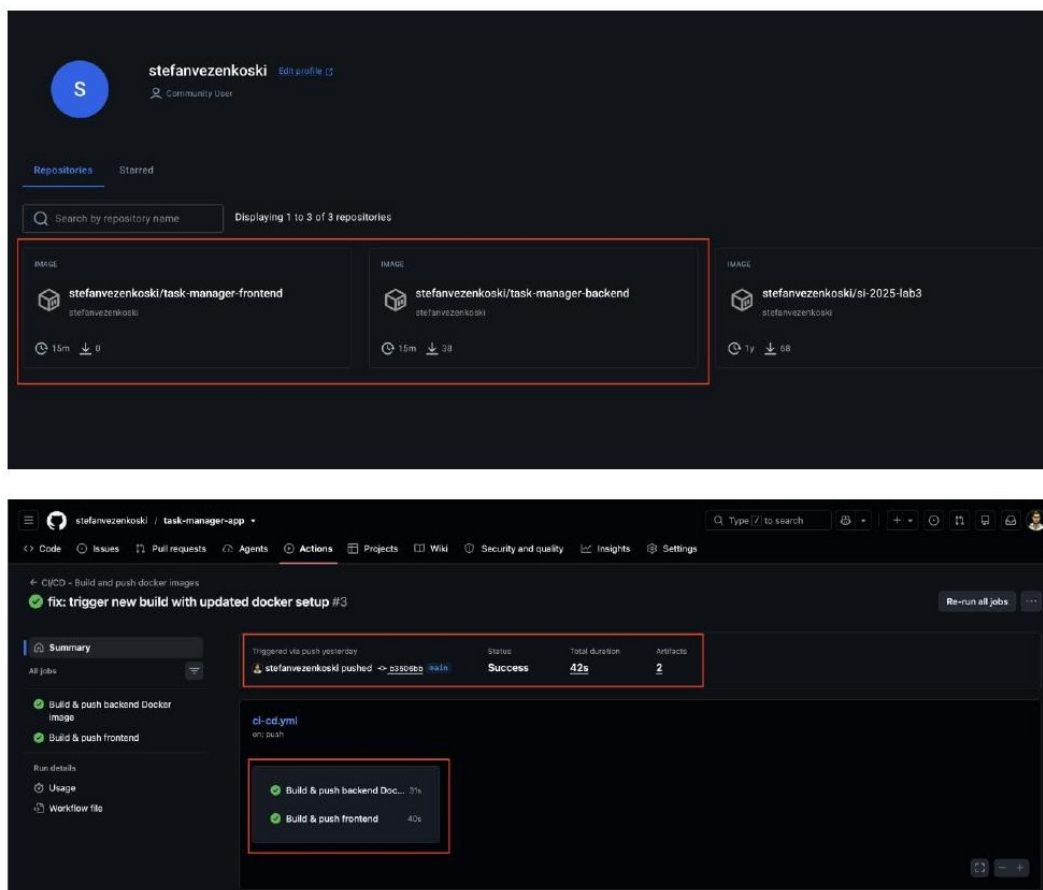
Избрана CI платформа: **GitHub Actions** — веќе интегрирана со GitHub (каде е хостиран кодот), без потреба од дополнителна инфраструктура за основниот CI дел, со голема библиотека на готови actions за Docker build/push.

Pipeline-от се активира при секој push на main гранката, и извршува два паралелни job-a — build и push на backend и frontend Docker имиците на DockerHub, секој означен со тагови latest и commit SHA за прецизен version tracking:

```
ci-cd.yml

1 name: CI/CD - Build and push docker images
2
3 on:
4   push:
5     branches: ['main']
6   pull_request:
7     branches: ['main']
8
9 env:
10  DOCKERHUB_USERNAME: ${ secrets.DOCKERHUB_USERNAME }
11  BACKEND_IMAGE: task-manager-backend
12  FRONTEND_IMAGE: task-manager-frontend
13
14 jobs:
15   build-and-push-backend:
16     name: Build & push backend Docker image
17     runs-on: ubuntu-latest
18     steps:
19       - name: Checkout code
20         uses: actions/checkout@v4
21
22       - name: Login to DockerHub
23         if: github.event_name != 'pull_request'
24         uses: docker/login-action@v3
25         with:
26           username: ${ secrets.DOCKERHUB_USERNAME }
27           password: ${ secrets.DOCKERHUB_TOKEN }
28
29       - name: Docker Buildx
30         uses: docker/setup-buildx-action@v3
31
32       - name: Build и push - backend image
33         uses: docker/build-push-action@v6
34         with:
35           context: ./backend
36           platforms: linux/amd64,linux/arm64
37           push: ${ github.event_name != 'pull_request' }
38           tags: |
39             ${ secrets.DOCKERHUB_USERNAME }/task-manager-backend:latest
40             ${ env.DOCKERHUB_USERNAME }/${ env.BACKEND_IMAGE }:${ github.sha }
41           cache-from: type=gha
42           cache-to: type=gha,mode=max
43
44   build-and-push-frontend:
45     name: Build & push frontend
46     runs-on: ubuntu-latest
47     steps:
48       - name: Checkout code
49         uses: actions/checkout@v4
50
51       - name: Login to DockerHub
52         if: github.event_name != 'pull_request'
53         uses: docker/login-action@v3
54         with:
55           username: ${ secrets.DOCKERHUB_USERNAME }
56           password: ${ secrets.DOCKERHUB_TOKEN }
57
58       - name: Docker Buildx
59         uses: docker/setup-buildx-action@v3
60
61       - name: Build и push - frontend image
62         uses: docker/build-push-action@v6
63         with:
64           context: ./frontend
65           platforms: linux/amd64,linux/arm64
66           push: ${ github.event_name != 'pull_request' }
67           tags: |
68             ${ env.DOCKERHUB_USERNAME }/${ env.FRONTEND_IMAGE }:latest
69             ${ env.DOCKERHUB_USERNAME }/${ env.FRONTEND_IMAGE }:${ github.sha }
70           cache-from: type=gha
71           cache-to: type=gha,mode=max
72
73   deploy:
74     name: Deploy to local k3d cluster
75     needs: [build-and-push-backend, build-and-push-frontend]
76     runs-on: self-hosted
77     if: github.ref == 'refs/heads/main'
78     defaults:
79       run:
80         shell: bash
81     steps:
82       - name: Checkout код
83         uses: actions/checkout@v4
84
85       - name: Restart deployments (новлекува нови :latest слики)
86         run: |
87           kubectl rollout restart deployment/backend -n task-manager
88           kubectl rollout restart deployment/frontend -n task-manager
89
90       - name: Провери статус на rollout
91         run: |
92           kubectl rollout status deployment/backend -n task-manager --timeout=90s
93           kubectl rollout status deployment/frontend -n task-manager --timeout=90s
94
95       - name: Финална проверка
96         run: kubectl get pods -n task-manager
```

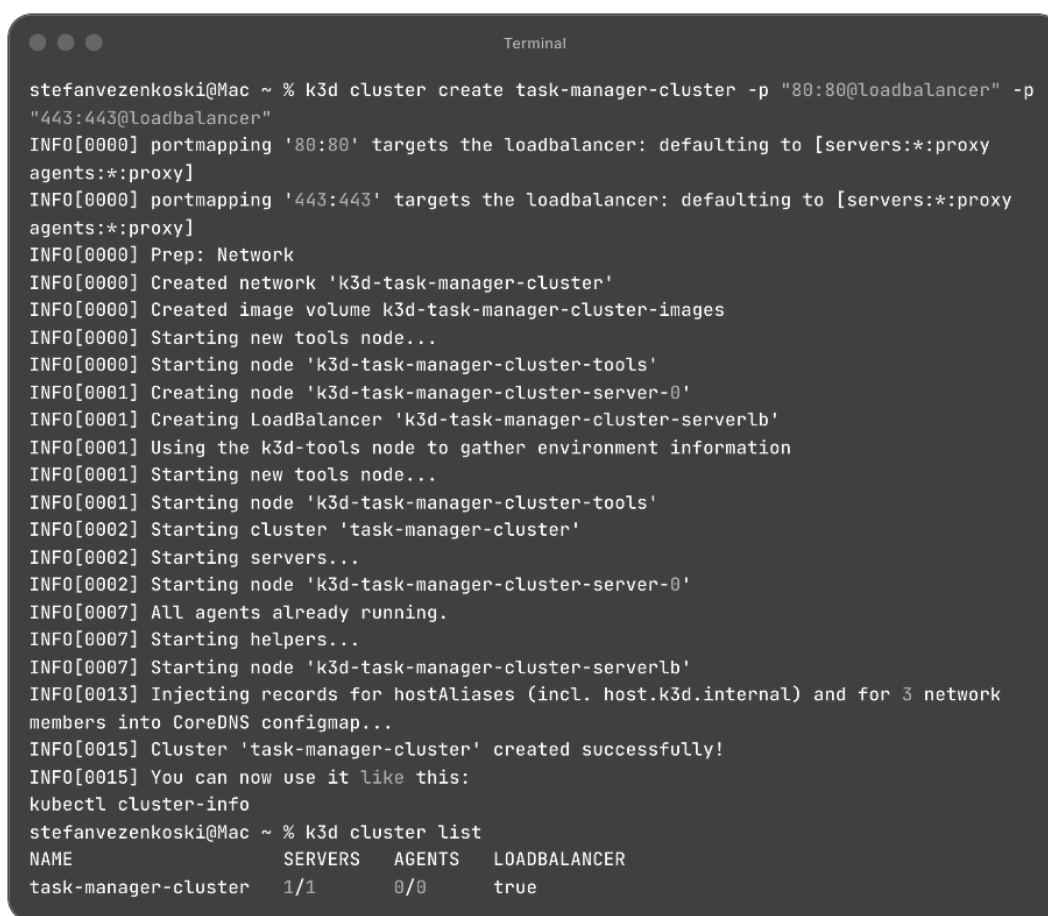
Слика 3.1: .github/workflows/ci-cd.yml



Слика 3.2: Push-нати имици на DockerHub и успешен GitHub Actions run

Kubernetes манифести

Кластерот е поставен локално со k3d (лесен K3s дистрибуција во Docker), кој доаѓа со Traefik како вграден Ingress Controller. Сите манифести се рачно напишани (без Helm charts или operators), согласно барањето на предметот.

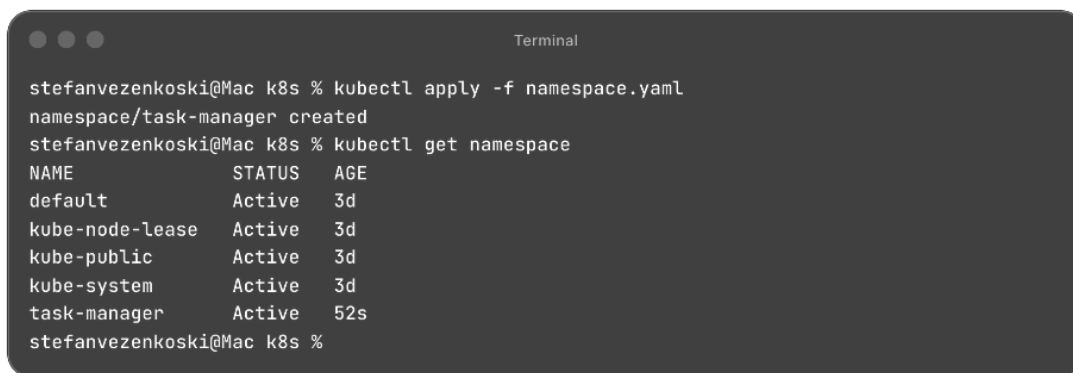


```
stefanvezenkoski@Mac ~ % k3d cluster create task-manager-cluster -p "80:80@loadbalancer" -p "443:443@loadbalancer"
INFO[0000] portmapping '80:80' targets the loadbalancer: defaulting to [servers::proxy agents::proxy]
INFO[0000] portmapping '443:443' targets the loadbalancer: defaulting to [servers::proxy agents::proxy]
INFO[0000] Prep: Network
INFO[0000] Created network 'k3d-task-manager-cluster'
INFO[0000] Created image volume k3d-task-manager-cluster-images
INFO[0000] Starting new tools node...
INFO[0000] Starting node 'k3d-task-manager-cluster-tools'
INFO[0001] Creating node 'k3d-task-manager-cluster-server-0'
INFO[0001] Creating LoadBalancer 'k3d-task-manager-cluster-serverlb'
INFO[0001] Using the k3d-tools node to gather environment information
INFO[0001] Starting new tools node...
INFO[0001] Starting node 'k3d-task-manager-cluster-tools'
INFO[0002] Starting cluster 'task-manager-cluster'
INFO[0002] Starting servers...
INFO[0002] Starting node 'k3d-task-manager-cluster-server-0'
INFO[0007] All agents already running.
INFO[0007] Starting helpers...
INFO[0007] Starting node 'k3d-task-manager-cluster-serverlb'
INFO[0013] Injecting records for hostAliases (incl. host.k3d.internal) and for 3 network members into CoreDNS configmap...
INFO[0015] Cluster 'task-manager-cluster' created successfully!
INFO[0015] You can now use it like this:
kubectl cluster-info
stefanvezenkoski@Mac ~ % k3d cluster list
NAME          SERVERS  AGENTS  LOADBALANCER
task-manager-cluster  1/1      0/0      true
```

Слика 4.1: Креирање на k3d кластерот

6.1 Namespace

Сите ресурси на проектот се изолирани во посебен namespace task-manager, одделен од системските namespace-и на кластерот:



```
stefanvezenkoski@Mac k8s % kubectl apply -f namespace.yaml
namespace/task-manager created
stefanvezenkoski@Mac k8s % kubectl get namespace
NAME                STATUS    AGE
default              Active   3d
kube-node-lease      Active   3d
kube-public          Active   3d
kube-system          Active   3d
task-manager         Active   52s
stefanvezenkoski@Mac k8s %
```

Слика 4.2: namespace.yaml, kubectl apply и kubectl get namespace

6.2 StatefulSet за базата (со ConfigMap и Secret)

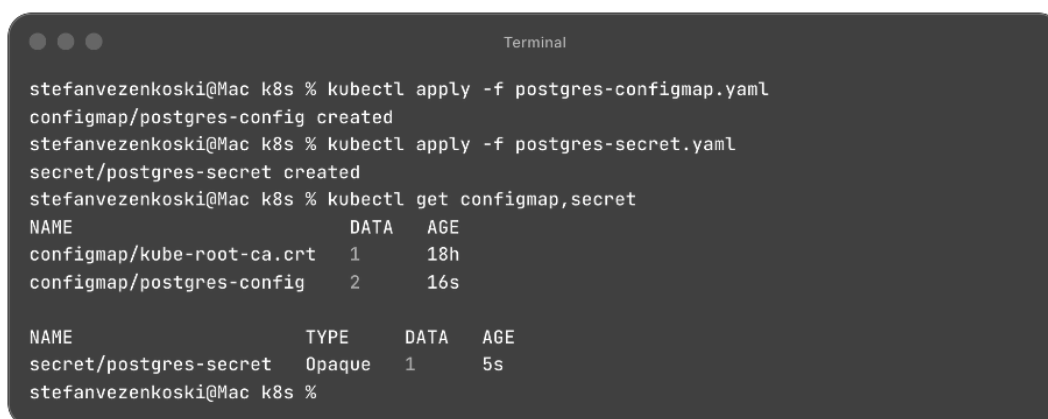
PostgreSQL базата бара **персистентно складиште** и стабилен мрежен идентитет, што StatefulSet (за разлика од Deployment) го обезбедува преку volumeClaimTemplates. Конфигурацијата (корисник, име на база) е одделена во ConfigMap, а лозинката во Secret:



```
1 apiVersion: v1
2 kind: ConfigMap
3 metadata:
4   name: postgres-config
5   namespace: task-manager
6 data:
7   POSTGRES_USER: postgres
8   POSTGRES_DB: taskdb

1 apiVersion: v1
2 kind: Secret
3 metadata:
4   name: postgres-secret
5   namespace: task-manager
6 type: Opaque
7 stringData:
8   POSTGRES_PASSWORD: postgres
```

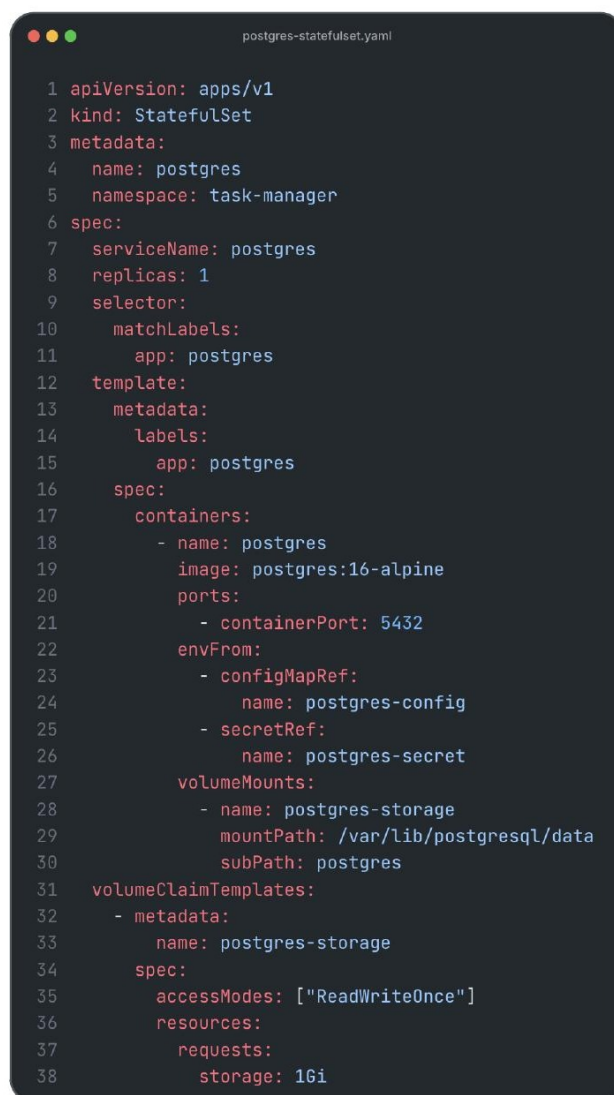
Слика 4.3: postgres-configmap.yaml и postgres-secret.yaml



```
stefanvezenkoski@Mac k8s % kubectl apply -f postgres-configmap.yaml
configmap/postgres-config created
stefanvezenkoski@Mac k8s % kubectl apply -f postgres-secret.yaml
secret/postgres-secret created
stefanvezenkoski@Mac k8s % kubectl get configmap,secret
NAME                                DATA  AGE
configmap/kube-root-ca.crt          1      18h
configmap/postgres-config            2      16s

NAME                                TYPE    DATA  AGE
secret/postgres-secret               Opaque  1       5s
stefanvezenkoski@Mac k8s %
```

Слика 4.4: Верификација: ConfigMap и Secret успешно креирани во кластерот



```
1 apiVersion: apps/v1
2 kind: StatefulSet
3 metadata:
4   name: postgres
5   namespace: task-manager
6 spec:
7   serviceName: postgres
8   replicas: 1
9   selector:
10    matchLabels:
11      app: postgres
12   template:
13     metadata:
14       labels:
15         app: postgres
16     spec:
17       containers:
18       - name: postgres
19         image: postgres:16-alpine
20         ports:
21         - containerPort: 5432
22         envFrom:
23         - configMapRef:
24             name: postgres-config
25         - secretRef:
26             name: postgres-secret
27       volumeMounts:
28       - name: postgres-storage
29         mountPath: /var/lib/postgresql/data
30         subPath: postgres
31   volumeClaimTemplates:
32   - metadata:
33       name: postgres-storage
34     spec:
35       accessModes: ["ReadWriteOnce"]
36       resources:
37         requests:
38           storage: 16Gi
```

Слика 4.5: postgres-statefulset.yaml

```

Terminal
stefanvezenkoski@Mac k8s % kubectl apply -f postgres-statefulset.yaml

statefulset.apps/postgres created
stefanvezenkoski@Mac k8s % kubectl get pods -n task-manager -w
NAME          READY   STATUS             RESTARTS   AGE
postgres-0    0/1     ContainerCreating   0           35s
postgres-0    1/1     Running             0           64s

Terminal
stefanvezenkoski@Mac k8s % kubectl get pvc -n task-manager
NAME                                STATUS   VOLUME                                     CAPACITY
ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
postgres-storage-postgres-0    Bound    pvc-29aafdf3-96fd-4bc4-80d4-00349de6893d    16i
RW0             local-path     <unset>                               8m
stefanvezenkoski@Mac k8s %

```

Слика 4.6: Подот postgres-0 стартува и станува Running; PersistentVolumeClaim е Bound (1Gi)

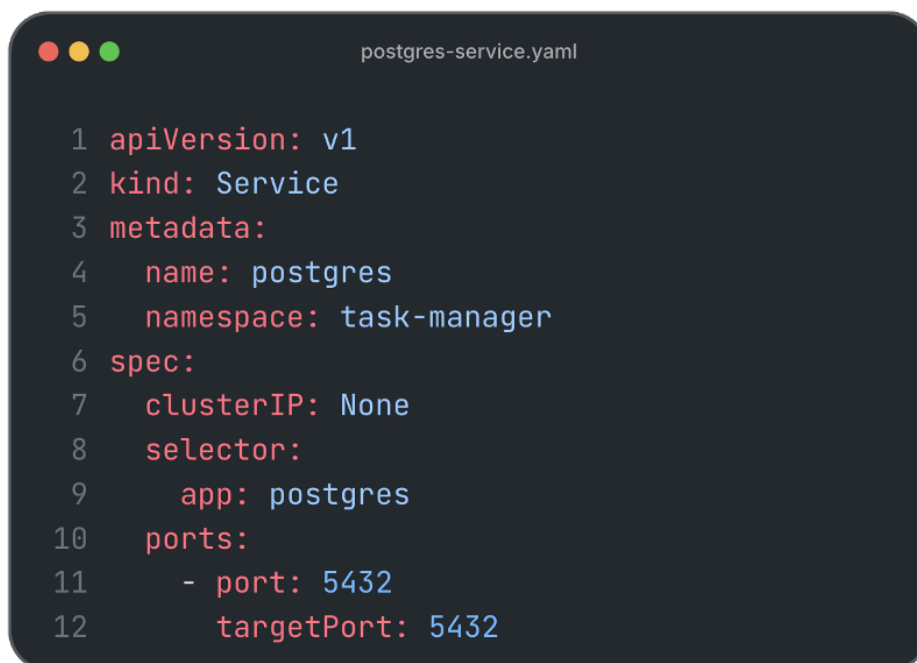
Забелешка за репликација на базата

StatefulSet-от е конфигуриран со replicas: 1 намерно. Зголемување на replicas за PostgreSQL **не** создава автоматска репликација на податоците — секој под во StatefulSet добива сопствен, независен PersistentVolumeClaim преку volumeClaimTemplates, што значи дополнителни реплики би започнале со **празна** база, несинхронизирана со примарната. Вистинска PostgreSQL репликација бара рачно конфигурирана streaming replication (primary/standby, pg_basebackup, WAL streaming) — значително понапредна тема од опфатот на предметот. Со еден реплика, критериумот „StatefulSet за базата со потребните ConfigMaps/Secrets“ е целосно исполнет, без ризик од демонстрирање нескладни податоци меѓу инстанци.

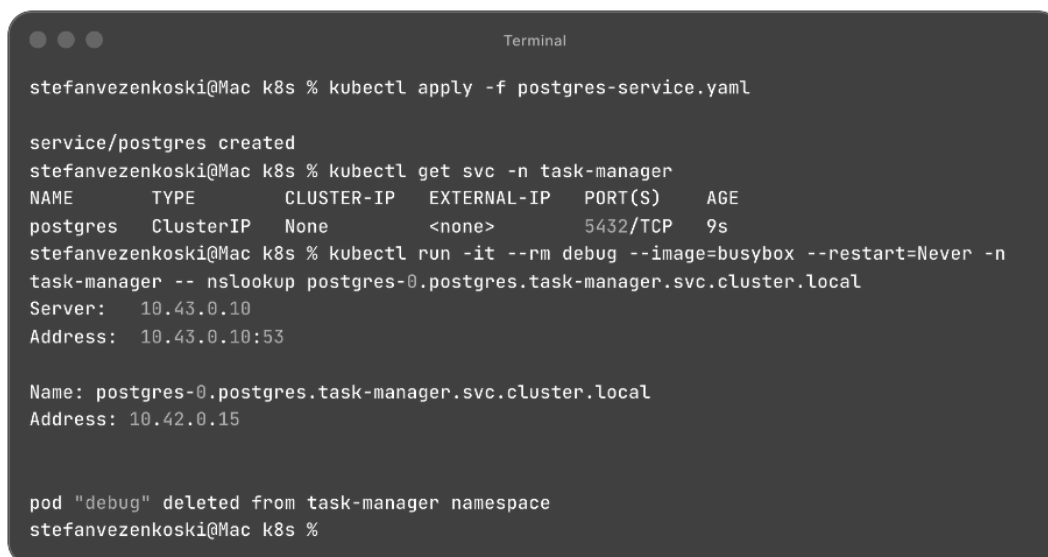
6.3 Headless Service за базата (postgres-service.yaml)

За StatefulSet-от да функционира правилно, потребен е headless Service (clusterIP: None), поврзан со StatefulSet-от преку serviceName: postgres. За разлика од обичен Service, кој дава едно заедничко IP и балансира сообраќај меѓу подовите, headless Service дава стабилно DNS име за секој под посебно — важно за база на податоци, каде мора да се знае точно со кој под се комуницира.

kubectl get svc потврдува дека Service-от е креиран со CLUSTER-IP: None (headless). Тест со привремен под (nslookup) потврдува дека DNS резолуцијата работи правилно — postgres-0.postgres.task-manager.svc.cluster.local успешно се преведува во внатрешна IP адреса на кластерот. Ова DNS име е тоа што backend-от подоцна го користи во DATABASE_URL за да се поврзе со базата.



Слика 4.7: Headless Service манифест за PostgreSQL (postgres-service.yaml)

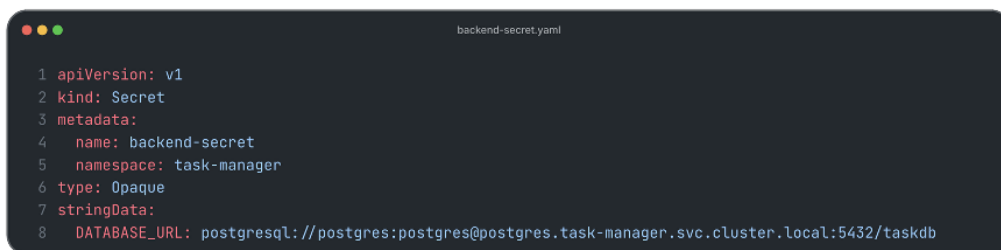


Слика 4.8: Креирање и верификација на headless Service (kubectl + nslookup)

6.4 Secret за backend (backend-secret.yaml)

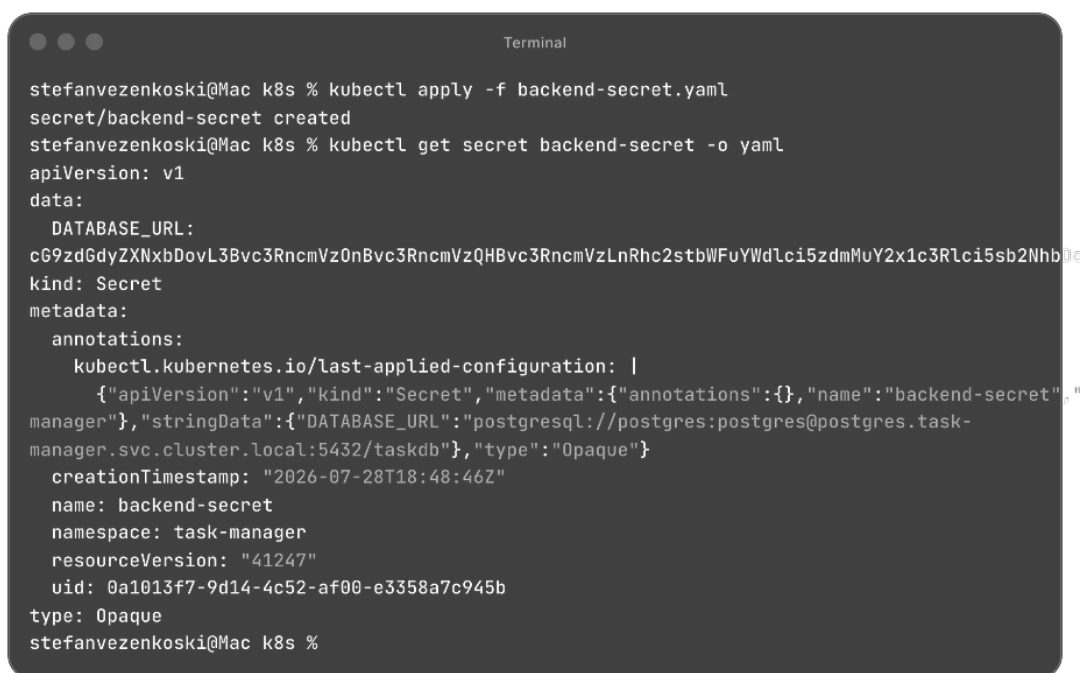
Конекцијата кон базата (DATABASE_URL) се чува во посебен Secret ресурс, одделно од Docker имиџот, и се инјектира во backend подовите како environment variable. Ова е стандардна пракса — чувствителните податоци (корисничко име, лозинка, host, порт, име на база) никогаш не се hardcode-ираат директно во кодот или во Docker имиџот.

Kubernetes автоматски го base64-енкодира текстот од stringData при чување на Secret ресурсот во кластерот (затоа во YAML излезот наместо postgresql://... се гледа енкодиран стринг) — ова не е вистинска енкрипција, туку стандарден формат за пренос на Secret вредности.



```
1 apiVersion: v1
2 kind: Secret
3 metadata:
4   name: backend-secret
5   namespace: task-manager
6 type: Opaque
7 stringData:
8   DATABASE_URL: postgresql://postgres:postgres@postgres.task-manager.svc.cluster.local:5432/taskdb
```

Слика 4.9: Secret ресурс со DATABASE_URL (backend-secret.yaml)



```
stefanvezenkoski@Mac k8s % kubectl apply -f backend-secret.yaml
secret/backend-secret created
stefanvezenkoski@Mac k8s % kubectl get secret backend-secret -o yaml
apiVersion: v1
data:
  DATABASE_URL:
    cG9zdGdyZXNxbDovL3Bvc3RncmVzOnBvc3RncmVzQHBvc3RncmVzLnRhczstbWYuYWdlci5zdmMuY2x1c3Rlcj5sb2NhbnQe
kind: Secret
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"Secret","metadata":{"annotations":{"name":"backend-secret"},"stringData":{"DATABASE_URL":"postgresql://postgres:postgres@postgres.task-manager.svc.cluster.local:5432/taskdb"},"type":"Opaque"},"creationTimestamp":"2026-07-28T18:48:46Z"}
  creationTimestamp: "2026-07-28T18:48:46Z"
  name: backend-secret
  namespace: task-manager
  resourceVersion: "41247"
  uid: 0a1013f7-9d14-4c52-af00-e3358a7c945b
type: Opaque
stefanvezenkoski@Mac k8s %
```

Слика 4.10: Креирање и верификација на Secret ресурсот во кластерот

6.5 Deployment за backend (backend-deployment.yaml)

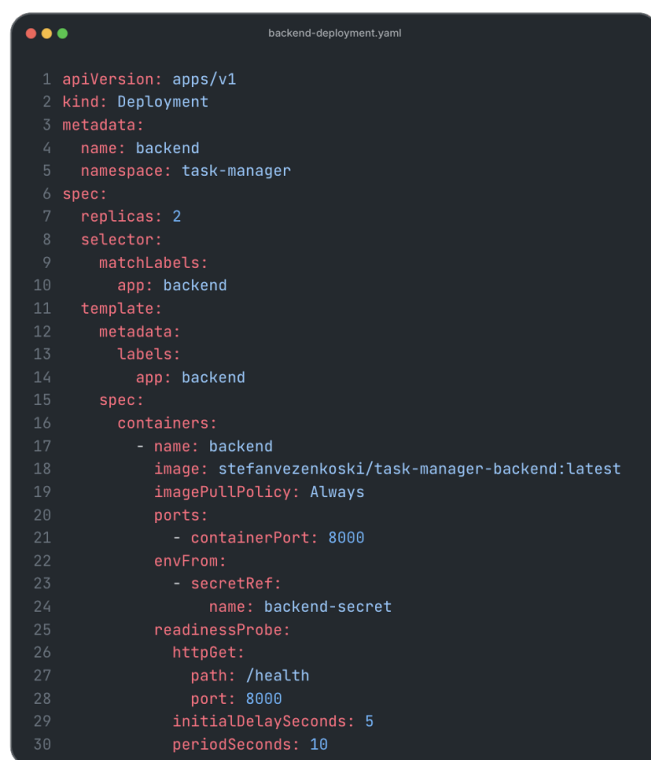
Дефиниран е Kubernetes Deployment манифест за backend апликацијата, во рамки на изолираниот task-manager namespace.

Клучни делови од конфигурацијата:

- **Репликација:** replicas: 2 — бекенд апликацијата се извршува на два паралелни пода, овозможувајќи балансирање на сообраќајот и отпорност на откази (High Availability).

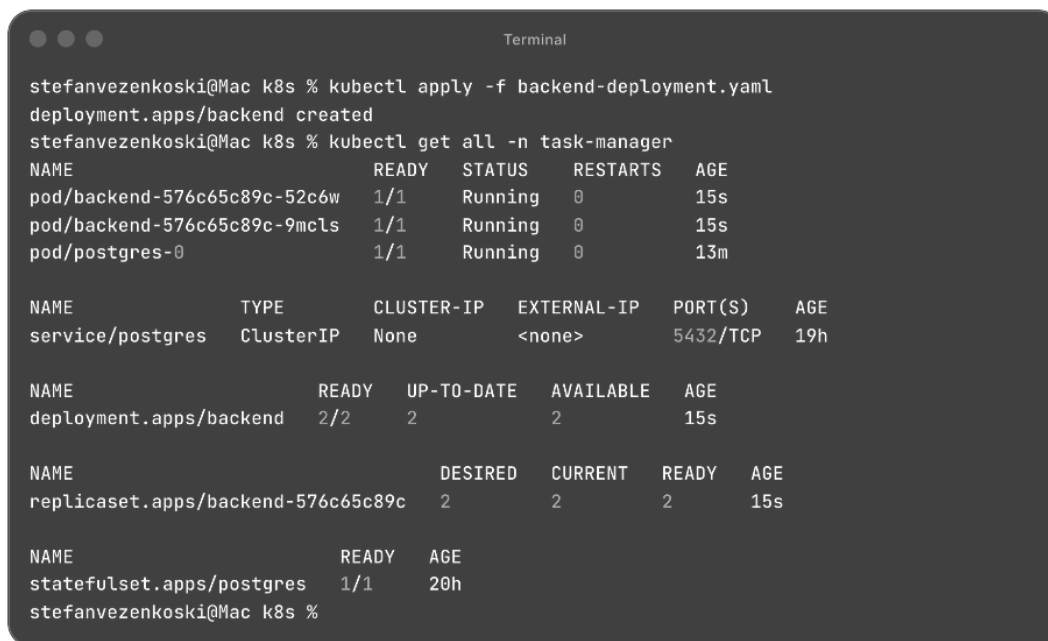
- **Docker имиџ:** се користи официјалната слика повлечена од DockerHub (`stefanvezenkoski/task-manager-backend:latest`).
- **imagePullPolicy:** `Always` — гарантира дека при секое стартување/рестартирање Kubernetes ќе ја повлече најновата верзија од регистарот.
- **Управување со тајни:** преку `envFrom` и `secretRef: name: backend-secret`, конфигурацијата за базата се вбризгува безбедно, без `hardcode`-ирани вредности во манифестот.
- **Health checks:** `readinessProbe` автоматски проверува подготвеност преку HTTP GET на `/health`, со почетна пауза од 5 секунди и интервал од 10 секунди — Kubernetes пренасочува сообраќај само кон целосно иницијализирани подови.

По `apply`-ирање, проверката со `kubectl get all` потврдува дека двата `backend` пода се `Running`, а `Deployment`-от е во состојба `2/2 AVAILABLE`.



```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: backend
5   namespace: task-manager
6 spec:
7   replicas: 2
8   selector:
9     matchLabels:
10      app: backend
11   template:
12     metadata:
13       labels:
14         app: backend
15     spec:
16       containers:
17       - name: backend
18         image: stefanvezenkoski/task-manager-backend:latest
19         imagePullPolicy: Always
20         ports:
21         - containerPort: 8000
22         envFrom:
23         - secretRef:
24             name: backend-secret
25         readinessProbe:
26           httpGet:
27             path: /health
28             port: 8000
29           initialDelaySeconds: 5
30           periodSeconds: 10
```

Слика 4.11: Deployment манифест за backend (backend-deployment.yaml)



```

stefanvezenkoski@Mac k8s % kubectl apply -f backend-deployment.yaml
deployment.apps/backend created
stefanvezenkoski@Mac k8s % kubectl get all -n task-manager
NAME                                READY   STATUS    RESTARTS   AGE
pod/backend-576c65c89c-52c6w        1/1     Running   0           15s
pod/backend-576c65c89c-9mcl5        1/1     Running   0           15s
pod/postgres-0                       1/1     Running   0           13m

NAME                                TYPE          CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
service/postgres                     ClusterIP     None         <none>         5432/TCP   19h

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/backend              2/2     2             2           15s

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/backend-576c65c89c  2         2         2       15s

NAME                                READY   AGE
statefulset.apps/postgres           1/1     20h
stefanvezenkoski@Mac k8s %

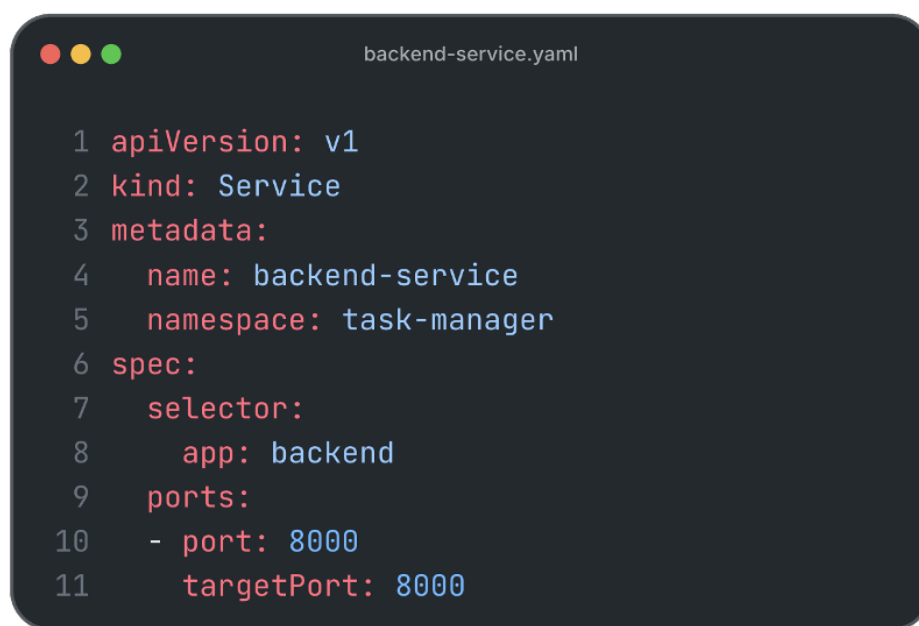
```

Слика 4.12: Верификација: backend Deployment (2 реплики) е успешно креиран и работи

6.6 Service за backend (backend-service.yaml)

За стабилна внатрешна комуникација помеѓу компонентите во кластерот (фронен-дот и бекендот), дефиниран е Service ресурс.

Преку селекторот `app: backend`, сервисот динамички ги открива и поврзува сите активни backend подови управувани од Deployment-от. Бидејќи не е дефиниран друг тип, Kubernetes го креира како ClusterIP — виртуелна интерна IP адреса која служи како единствена влезна точка и балансира сообраќај помеѓу двете реплики на бекендот.

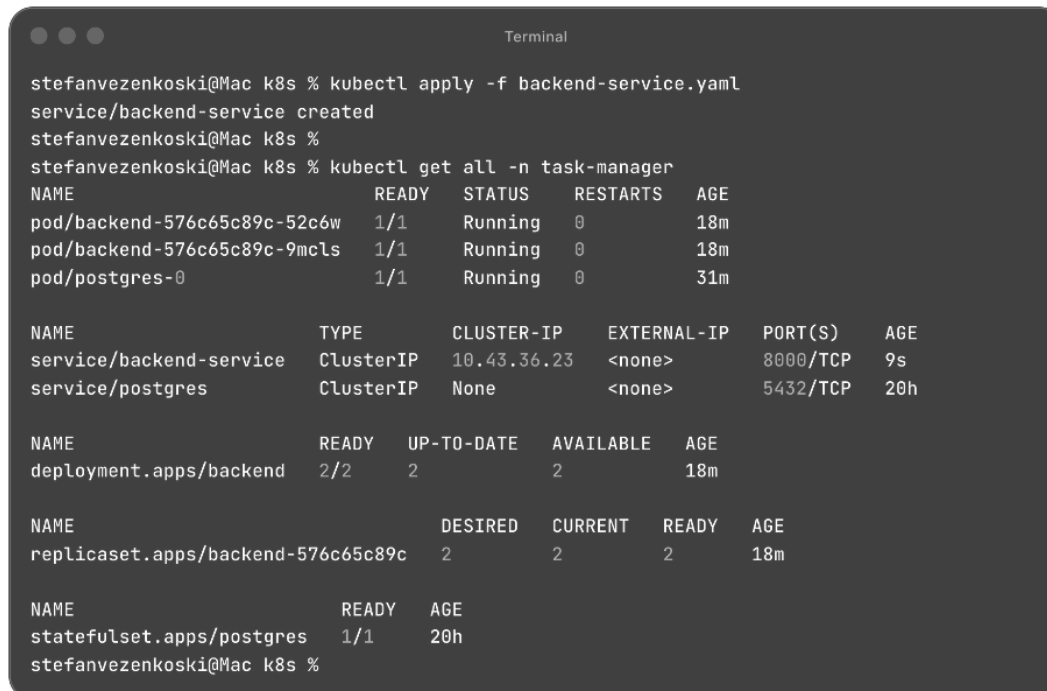


```

1 apiVersion: v1
2 kind: Service
3 metadata:
4   name: backend-service
5   namespace: task-manager
6 spec:
7   selector:
8     app: backend
9   ports:
10  - port: 8000
11    targetPort: 8000

```

Слика 4.13: Service ресурс за backend (backend-service.yaml)



```

stefanvezenkoski@Mac k8s % kubectl apply -f backend-service.yaml
service/backend-service created
stefanvezenkoski@Mac k8s %
stefanvezenkoski@Mac k8s % kubectl get all -n task-manager
NAME                                READY   STATUS    RESTARTS   AGE
pod/backend-576c65c89c-52c6w        1/1     Running   0           18m
pod/backend-576c65c89c-9mc1s        1/1     Running   0           18m
pod/postgres-0                      1/1     Running   0           31m

NAME                                TYPE               CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
service/backend-service             ClusterIP          10.43.36.23  <none>        8000/TCP   9s
service/postgres                   ClusterIP          None         <none>        5432/TCP   20h

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/backend             2/2     2            2           18m

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/backend-576c65c89c  2         2         2       18m

NAME                                READY   AGE
statefulset.apps/postgres           1/1     20h
stefanvezenkoski@Mac k8s %

```

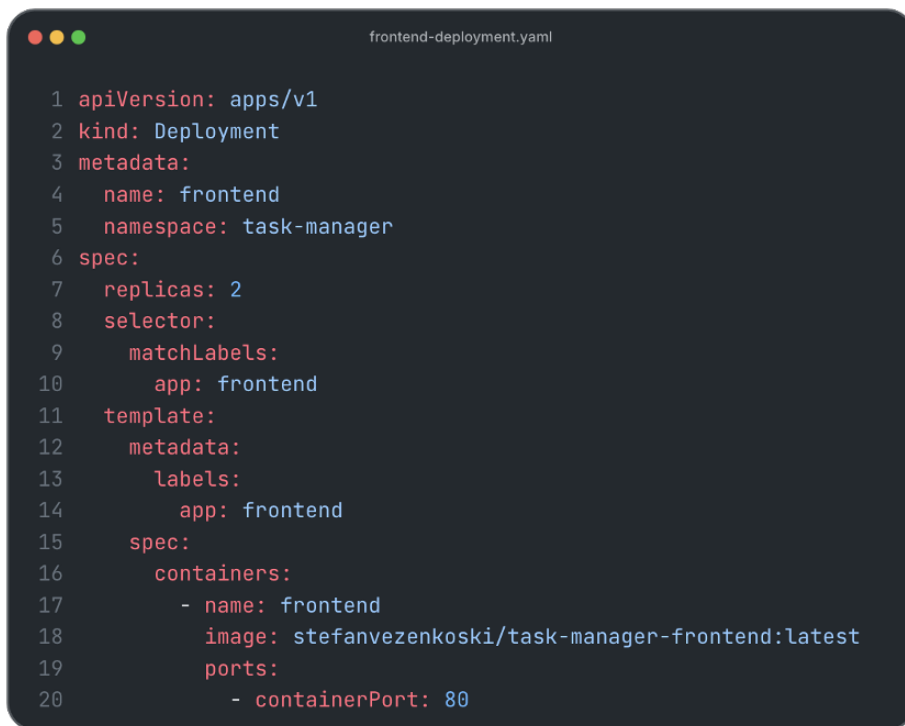
Слика 4.14: Верификација: backend Service е успешно креиран во namespace-от task-manager

6.7 Deployment за frontend (frontend-deployment.yaml)

За управување со корисничкиот интерфејс, дефиниран е Deployment манифест со

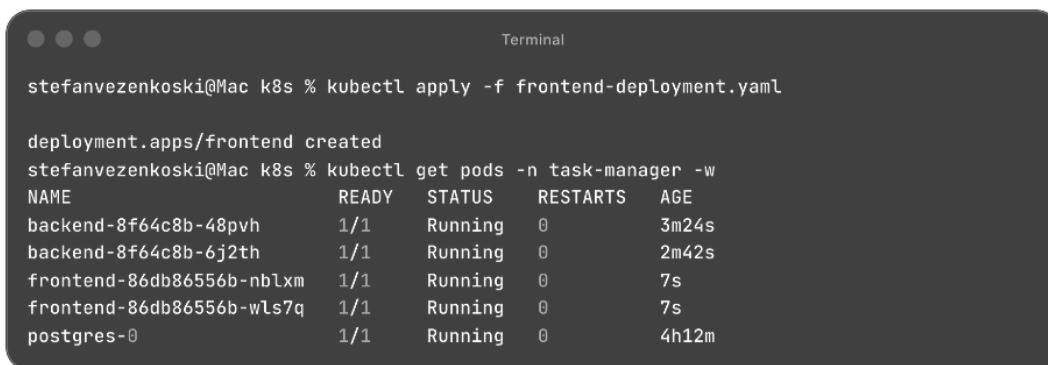
replicas: 2 за висока достапност, користејќи ја production Docker сликата (nginx + build-нат React код).

По apply-ирање, се создаваат два нови frontend пода, а Deployment-от достигнува статус 2/2 AVAILABLE.

A screenshot of a code editor window titled 'frontend-deployment.yaml'. The code is a Kubernetes Deployment manifest for a frontend application. It specifies 2 replicas, uses the 'stefanvezenkoski/task-manager-frontend:latest' image, and maps port 80 of the container to port 80 of the pod. The manifest is structured with 'apiVersion', 'kind', 'metadata', 'spec', and 'template' sections.

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: frontend
5   namespace: task-manager
6 spec:
7   replicas: 2
8   selector:
9     matchLabels:
10      app: frontend
11   template:
12     metadata:
13       labels:
14         app: frontend
15     spec:
16       containers:
17       - name: frontend
18         image: stefanvezenkoski/task-manager-frontend:latest
19         ports:
20         - containerPort: 80
```

Слика 4.15: Deployment манифест за frontend (frontend-deployment.yaml)

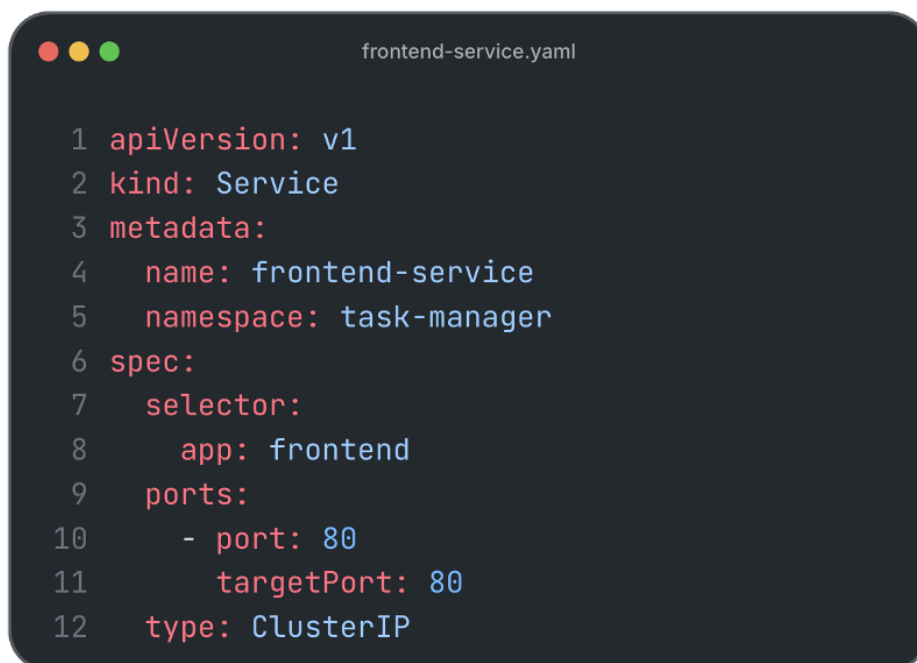
A screenshot of a terminal window titled 'Terminal'. It shows the command 'kubectl apply -f frontend-deployment.yaml' being executed successfully. Below the command output, the 'kubectl get pods -n task-manager -w' command is run, displaying a table of pods. The table shows five pods: two 'backend' pods and three 'frontend' pods, all in a 'Running' state with 1/1 ready replicas.

```
stefanvezenkoski@Mac k8s % kubectl apply -f frontend-deployment.yaml
deployment.apps/frontend created
stefanvezenkoski@Mac k8s % kubectl get pods -n task-manager -w
NAME                                READY   STATUS    RESTARTS   AGE
backend-8f64c8b-48pvh               1/1     Running   0           3m24s
backend-8f64c8b-6j2th               1/1     Running   0           2m42s
frontend-86db86556b-nblxm           1/1     Running   0           7s
frontend-86db86556b-wls7q           1/1     Running   0           7s
postgres-0                          1/1     Running   0           4h12m
```

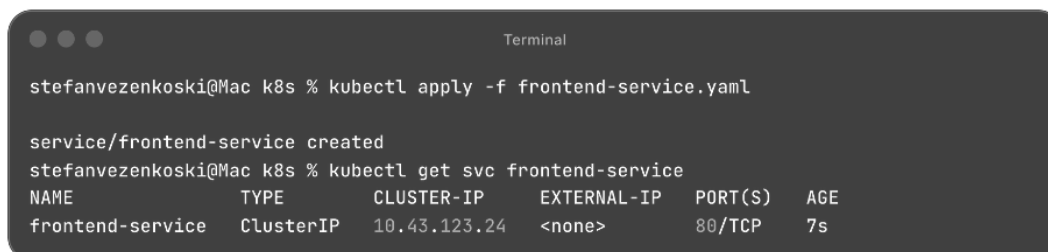
Слика 4.16: Верификација: frontend Deployment (2 реплики) е успешно креиран и работи

6.8 Service за frontend (frontend-service.yaml)

Истата логика како backend-service.yaml, но со селектор app: frontend и порт 80 (наместо 8000), усогласен со nginx контејнерот кој ги сервира статичките React фајлови.



Слика 4.17: Service ресурс за frontend (frontend-service.yaml)



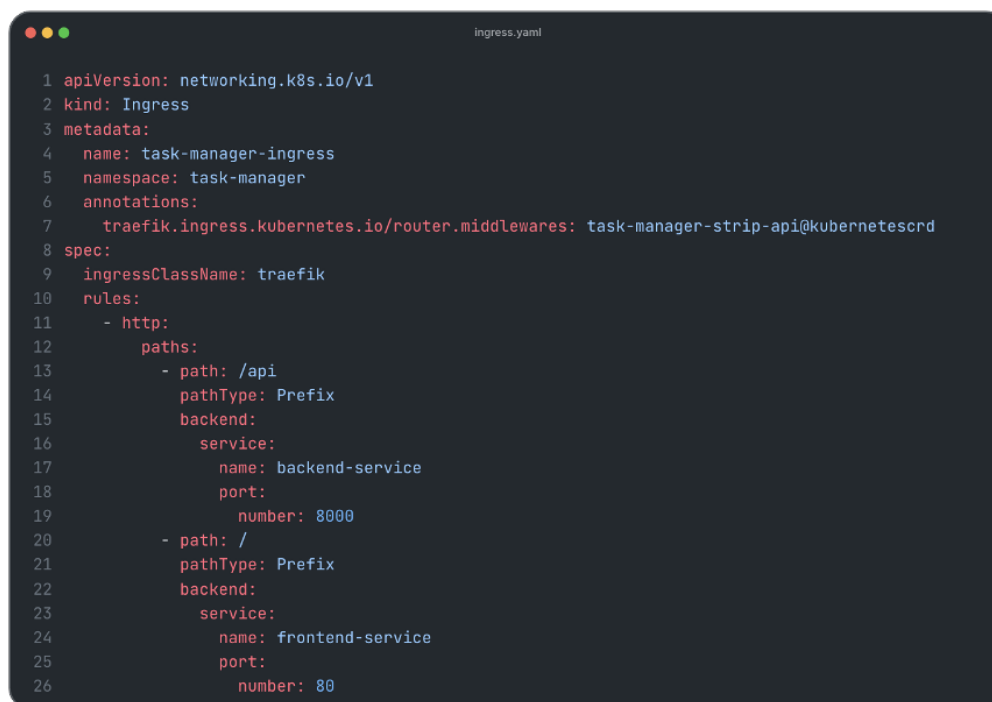
Слика 4.18: Верификација: frontend Service е успешно креиран во namespace-от task-manager

6.9 Ingress (ingress.yaml)

За надворешен пристап преку една единствена влезна точка (Reverse Proxy / Load Balancer), дефиниран е Ingress ресурс.

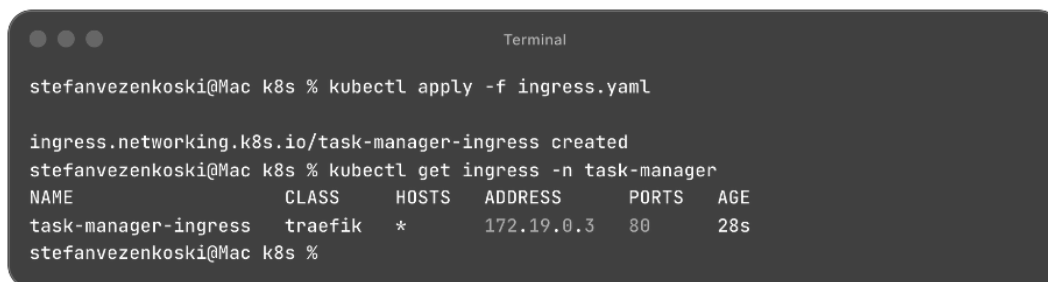
- **Ingress Class:** traefik — вграденото рутирање во k3d кластерот.
- **Middleware анотација:** го отстранува предфиксот /api од патеката пред барањето да биде препратено до бекендот.
- **Правила за рутирање (path-based routing):** барања на /api се насочуваат кон backend-service на порт 8000; сите останати барања (/) се насочуваат кон frontend-service на порт 80.

По apply-ирање, `kubectl get ingress` потврдува дека ресурсот е креиран, со CLASS `traefik`, HOSTS `*`, доделена ADDRESS, и порт 80.



```
1 apiVersion: networking.k8s.io/v1
2 kind: Ingress
3 metadata:
4   name: task-manager-ingress
5   namespace: task-manager
6   annotations:
7     traefik.ingress.kubernetes.io/router.middlewares: task-manager-strip-api@kubernetescrd
8 spec:
9   ingressClassName: traefik
10  rules:
11    - http:
12      paths:
13        - path: /api
14          pathType: Prefix
15          backend:
16            service:
17              name: backend-service
18              port:
19                number: 8000
20        - path: /
21          pathType: Prefix
22          backend:
23            service:
24              name: frontend-service
25              port:
26                number: 80
```

Слика 4.19: Ingress манифест (ingress.yaml)



```
stefanvezenkoski@Mac k8s % kubectl apply -f ingress.yaml

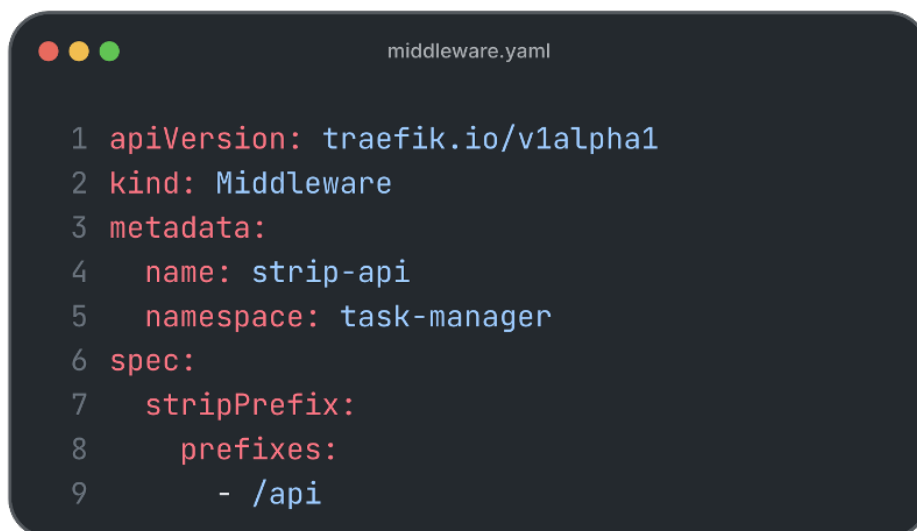
ingress.networking.k8s.io/task-manager-ingress created
stefanvezenkoski@Mac k8s % kubectl get ingress -n task-manager
NAME                CLASS    HOSTS    ADDRESS    PORTS    AGE
task-manager-ingress  traefik  *        172.19.0.3  80       28s
stefanvezenkoski@Mac k8s %
```

Слика 4.20: Верификација: Ingress ресурс е успешно креиран и има доделена адреса

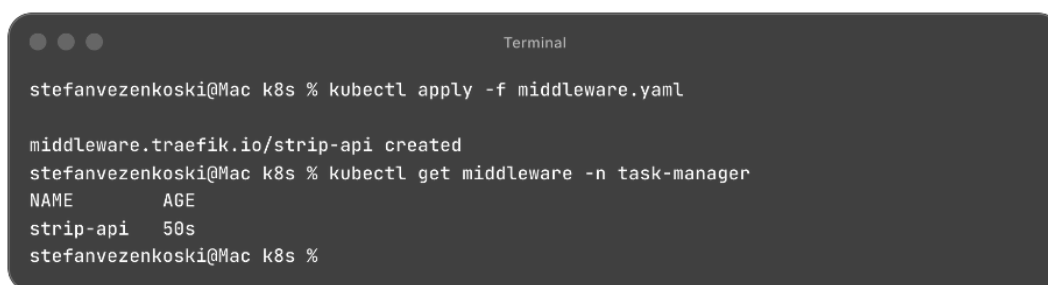
6.10 Traefik Middleware — стрипирање на /api префиксот (middleware.yaml)

Бидејќи фронтендот сите барања за податоци ги испраќа со предфикс `/api` (на пример `/api/tasks`), додека бекендот ги очекува рутите директно (на пример `/tasks`), потребен е Middleware ресурс кој автоматски го отстранува предфиксот.

Middleware е посредничка компонента во Traefik што ги пресретнува и модифицира HTTP барањата во реално време, пред тие да пристигнат до финалниот сервис. Правилото `stripPrefix: - /api` автоматски го отстранува `/api` од патеката, овозможувајќи му на бекендот да ги обработи барањата без 404 Not Found грешки.



Слика 4.21: Traefik Middleware за stripPrefix (middleware.yaml)



Слика 4.22: Верификација: Middleware ресурс е успешно креиран во кластерот

6.11 Демонстрација дека сè работи заедно

Целосен преглед на сите креирани ресурси во namespace-от task-manager одеднаш (kubectl get all,ingress,middleware -n task-manager) потврдува дека сите подови се Running, Deployment-ите се 2/2 AVAILABLE, StatefulSet-от е 1/1, а Ingress-от има доделена адреса.

Тест преку API, директно преку Ingress: curl http://localhost/api/health враќа {"status": "ok"} — ова докажува дека Ingress прима барање на /api/health, Middleware го отстранува /api, backend-от го гледа само /health, и одговара точно.

Тест преку browser на http://localhost/ — апликацијата се вчитува целосно (форма за нова задача, листа на задачи, статистика).

Целосен end-to-end тест: задача додадена преку browser UI, потврдена директно во базата преку kubectl exec -it postgres-0 -n task-manager -- psql -U postgres -d taskdb -c "SELECT * FROM tasks;" — доказ дека volumeClaimTemplates/persistent

storage реално работи.

Self-healing тест: `kubectl delete pod postgres-0 -n task-manager`, проследено со `kubectl get pods -n task-manager -w` — Kubernetes автоматски рекреира нов `postgres-0` под, кој повторно станува Running.

```

stefanvezenkoski@Mac k8s % kubectl get all,ingress,middleware -n task-manager
NAME                                READY   STATUS    RESTARTS   AGE
pod/backend-8f64c8b-48pvh           1/1     Running   0           19m
pod/backend-8f64c8b-6j2th           1/1     Running   0           18m
pod/frontend-86db86556b-nblxm       1/1     Running   0           15m
pod/frontend-86db86556b-wls7q       1/1     Running   0           15m
pod/postgres-0                      1/1     Running   0           4h28m

NAME                                TYPE               CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
service/backend-service             ClusterIP          10.43.81.126 <none>        8000/TCP    169m
service/frontend-service            ClusterIP          10.43.123.24 <none>        80/TCP      4m58s
service/postgres                    ClusterIP          None         <none>        5432/TCP    23h

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/backend             2/2     2             2           4h15m
deployment.apps/frontend            2/2     2             2           15m

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/backend-576c65c89c  0         0         0       4h15m
replicaset.apps/backend-8f64c8b     2         2         2       19m
replicaset.apps/frontend-86db86556b 2         2         2       15m

NAME                                READY   AGE
statefulset.apps/postgres           1/1     24h

NAME                                CLASS    HOSTS    ADDRESS    PORTS    AGE
ingress.networking.k8s.io/task-manager-ingress  traefik  *        172.19.0.3  80       2m30s

NAME                                AGE
middleware.traefik.io/strip-api     94s
stefanvezenkoski@Mac k8s %

```

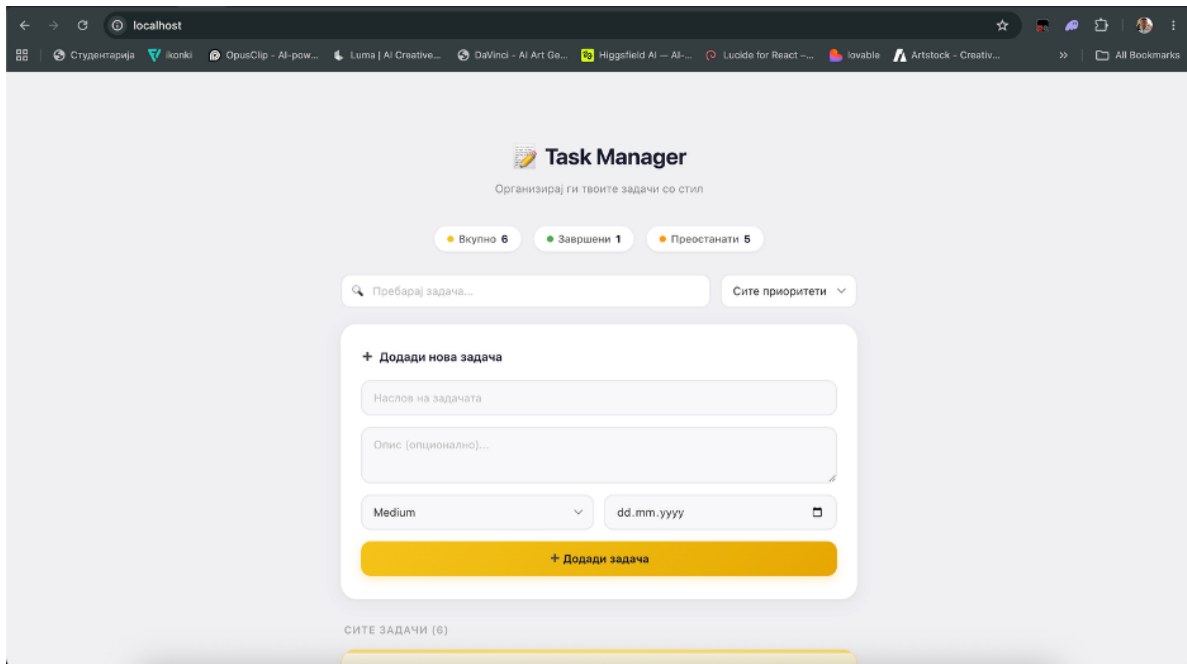
Слика 4.23: Целосен преглед на ресурси во task-manager namespace

```

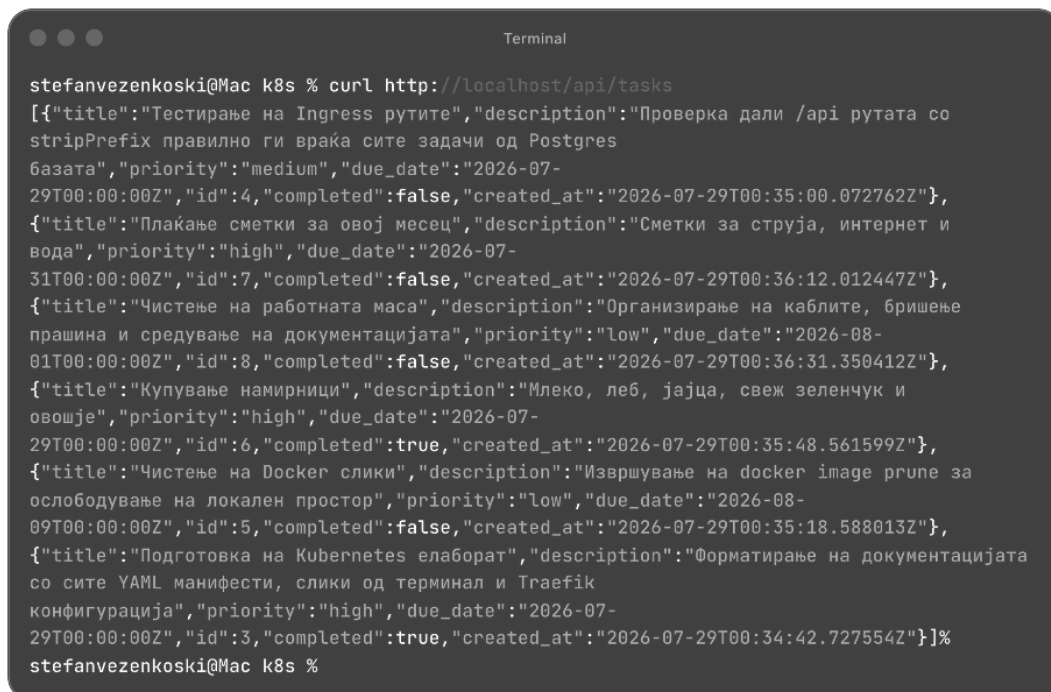
stefanvezenkoski@Mac k8s % curl http://localhost/api/health
{"status":"ok"}%
stefanvezenkoski@Mac k8s %

```

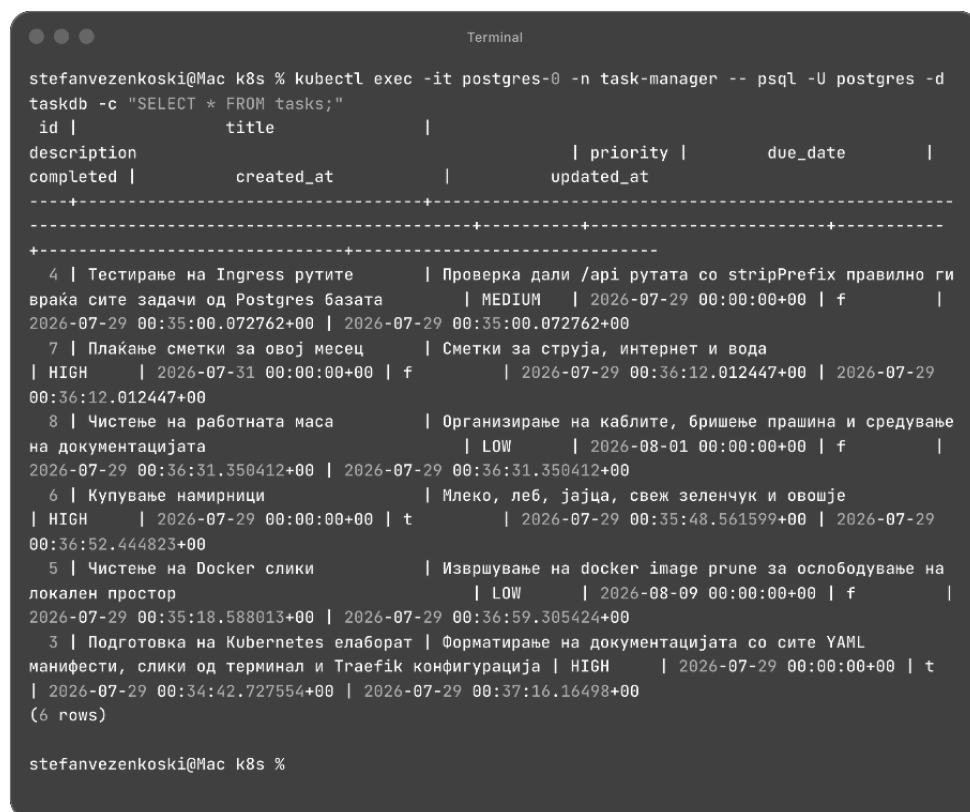
Слика 4.24: Проверка преку Ingress: /api/health враќа {"status": "ok"}



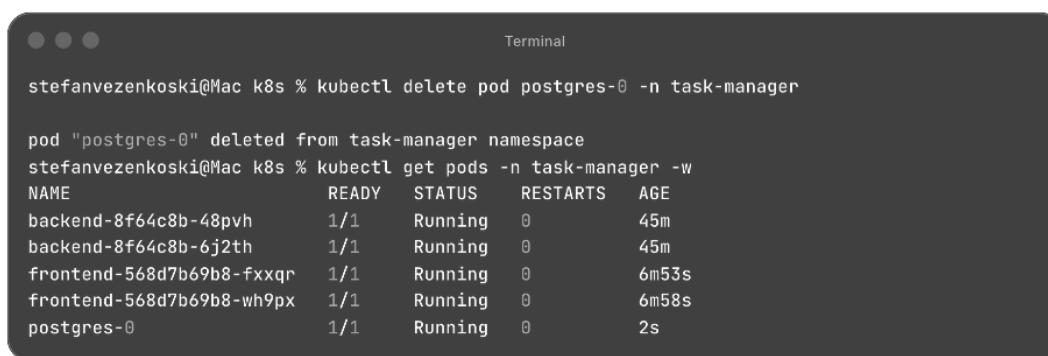
Слика 4.25: Frontend е достапен преку Ingress на `http://localhost/`



Слика 4.26: Проверка преку API: листа на задачи преку Ingress на `/api/tasks`



Слика 4.27: Доказ за перзистентност: `SELECT * FROM tasks;` во PostgreSQL



Слика 4.28: Self-healing: бришење на postgres-0 и автоматско рекреирање од StatefulSet

7. CD (бонус) — автоматски деплојмент

Бидејќи Kubernetes кластерот (k3d) работи локално, GitHub-хостираните runner-и (кои го извршуваат обичниот CI дел) немаат мрежен пристап до localhost на мојот компјутер. Затоа CD делот е реализиран преку self-hosted GitHub Actions runner — програма инсталирана директно на мојот Mac, регистрирана кон репозиториумот, која GitHub Actions може да ја „нареди“ да изврши деплојмент чекори локално.

Додаден е трет job (deploy) во истиот workflow, кој се извршува само на self-hosted runner-от, откако двата build job-а успешно ќе завршат. Тој извршува `kubectl rollout`

restart за backend и frontend Deployment-ите, со што се повлекуваат новите :latest имиџи од DockerHub автоматски при секој push.

Ограничување: CD делот бара self-hosted runner-от (`./run.sh`) да биде активен на мојот компјутер за да функционира — за разлика од CI делот, кој работи целосно независно на GitHub-овите сервери. Ова е нормално ограничување при работа со локален (не cloud-hosted) Kubernetes кластер.

```

Terminal
stefanvezenkoski@Mac task-manager-app % mkdir actions
stefanvezenkoski@Mac task-manager-app % cd actions
stefanvezenkoski@Mac actions % mkdir actions-runner && cd actions-runner
stefanvezenkoski@Mac actions-runner % curl -o actions-runner-osx-x64-2.336.0.tar.gz -L
https://github.com/actions/runner/releases/download/v2.336.0/actions-runner-osx-x64-
2.336.0.tar.gz
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0    0     0    0     0      0      0  --:--:-- --:--:-- --:--:--    0
100 125M 100 125M    0     0 1120k      0  0:01:54  0:01:54 --:--:-- 1151k
stefanvezenkoski@Mac actions-runner % echo
"f79c43232761ca495fc18df550bb2865aa99984b37c173c0aa1f8c089d0d548fe  actions-runner-osx-x64-
2.336.0.tar.gz" | shasum -a 256 -c
actions-runner-osx-x64-2.336.0.tar.gz: OK
stefanvezenkoski@Mac actions-runner % tar xzf ./actions-runner-osx-x64-2.336.0.tar.gz
stefanvezenkoski@Mac actions-runner % ./config.sh --url
https://github.com/stefanvezenkoski/task-manager-app --token BESKXDH65YQRXVSD360T6VDKNFTMY

-----
|
|      / _ _ \ | | | | | _ | | | | / \ _ _ \ | | | | _ _ _ _
| | | | | | | | | | | | | | \ / \ / \ | | | | / \ | ' \ / \ |
| | | | | | | | | | | | | | / _ _ \ | | | | | | | | | | \ \
| \ _ _ \ | \ _ _ \ | \ _ _ \ | \ / \ \ \ \ \ _ _ \ | | | | \
|
|                                     Self-hosted runner registration
|
|-----

# Authentication

✓ Connected to GitHub

# Runner Registration

Enter the name of the runner group to add this runner to: [press Enter for Default]

Enter the name of runner: [press Enter for Mac]

This runner will have the following labels: 'self-hosted', 'macOS', 'X64'
Enter any additional labels (ex. label-1,label-2): [press Enter to skip]

✓ Runner successfully added

# Runner settings

Enter name of work folder: [press Enter for _work]

✓ Settings Saved.

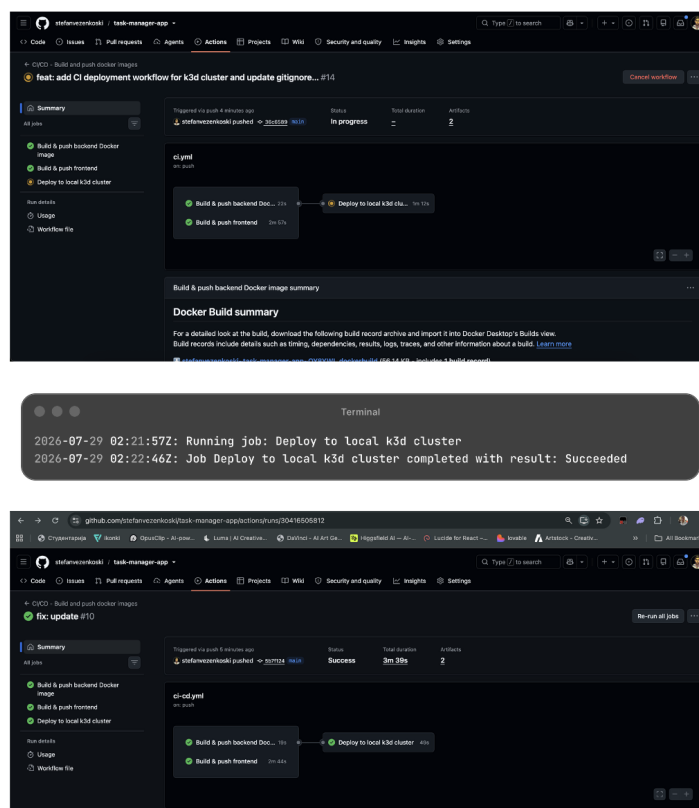
stefanvezenkoski@Mac actions-runner % ./run.sh

✓ Connected to GitHub

Current runner version: '2.336.0'
2026-07-29 01:53:04Z: Listening for Jobs

```

Слика 4.29: Self-hosted runner: регистрација и стартување (локален CD)



Слика 4.30: GitHub Actions pipeline со додаден deploy job на self-hosted runner

8. Проблеми и решенија

Конфликт на портови меѓу Docker и native PostgreSQL

Локален Homebrew PostgreSQL сервис (postgresql@14/@16) работеше паралелно со Docker контејнерот, обата обидувајќи се да го користат портот 5432. Решено со brew services stop за native сервисите, оставајќи Docker како единствен сопственик на портот.

npm ci паѓаше во GitHub Actions, а работеше локално

package-lock.json генериран на Apple Silicon (arm64) содржи platform-specific Rollup зависимости некомпатибилни со Alpine/musl Linux во Docker build контекстот. Решено со промена на build имиџот од node:20-alpine во node:20-slim (Debian-базиран).

Frontend прикажуваше празна страница зад Ingress

Првичниот frontend Dockerfile користеше Vite development server (npm run dev), чиј WebSocket за Hot Module Replacement не функционираше правилно низ Ingress. Решено со премин на production build (npm run build) сервиран преку nginx — стандардна практика за деплојмент на React апликации.

.gitignore не ги игнорираше Secret фајловите

Патеката во .gitignore не се совпаѓаше со реалното име на фајлот по преименување (стар префикс 04- останат во .gitignore по чистење на нумерираниите имиња на манифестите). Решено со усогласување на патеките и проверка преку git check-ignore -v.

database "taskdb" does not exist при поврзување од VS Code

Повеќе Docker контејнери и native PostgreSQL сервиси истовремено „се бореа“ за истиот порт 5432, па конекцијата понекогаш случајно се воспоставуваше кон сервер без базата taskdb. Решено со целосно чистење на сите конфликтни контејнери/сервиси и рестарт со единствен, чист Docker контејнер.

9. Заклучок

Проектот демонстрира целосен CI/CD и Kubernetes работен процес за реална, работечка three-tier апликација: од локален development (venv, npm, Docker Compose), преку автоматизиран CI pipeline (GitHub Actions → DockerHub), до целосна оркестрација и деплојмент во Kubernetes (Namespace, ConfigMap, Secret, StatefulSet со персистентно складиште, Deployment, Service, Ingress со Traefik Middleware), со опционален CD чекор преку self-hosted runner.

Низ изработката се стекнаа практични вештини за: Docker networking и debugging на конфликти на портови, YAML синтакса и Kubernetes ресурсни релации (Service ↔ StatefulSet, Ingress ↔ Middleware), разлика меѓу development и production конфигурации (Vite dev server наспроти nginx production build), и безбедно управување со чувствителни податоци (Secret фајлови одделени од Git преку .gitignore и .example шаблони).

Идеи за понатамошно проширување: имплементација на вистинска PostgreSQL streaming репликација (primary/standby), автентикација (JWT), автоматизирани тестови (pytest) интегрирани во CI pipeline-от, и премин кон cloud-hosted Kubernetes кластер за целосно независен CD без потреба од self-hosted runner.

10. Линкови

- **Git репозиториум:** <https://github.com/stefanvezenkoski/task-manager-app>
- **DockerHub — backend имиџ:** <https://hub.docker.com/r/stefanvezenkoski/task-manager-backend>
- **DockerHub — frontend имиџ:** <https://hub.docker.com/r/stefanvezenkoski/task-manager-frontend>